

Object-Oriented Programming in Java

Lecture 6 - Abstract Elements

Emily Lucia Antosch

HAW Hamburg

19.10.2025

Contents

1. Introduction	2
2. Abstract Elements & Methods	6
3. Interfaces	14
4. Comparison (Interface Comparable)	26
5. License Notice	34

1. Introduction

1.1 Where Are We Currently?

- The last lecture was about inheritance
- You can now
 - ▶ create and use simple inheritance lines,
 - ▶ override methods from the base class,
 - ▶ use the `equals()` method to compare objects with each other,
 - ▶ reference objects via the respective base class
- Today we continue with **Interfaces**.

1.1 Where Are We Currently?

1. Introduction

1. Imperative Concepts
2. Classes and Objects
3. Class Library
4. Inheritance
5. **Interfaces**
6. Graphical User Interfaces
7. Exception Handling
8. Input and Output
9. Multithreading (Parallel Computing)

1.2 The Goal of This Chapter

- You model common properties of classes by extending classes with common interfaces (in the form of abstract base classes or interfaces).
- You hide the data type of objects by referencing objects via interfaces when accessing common properties of different classes.
- You sort a collection of objects of the same data type according to arbitrary criteria

2. Abstract Elements & Methods

? Question

- Do you remember our geometric objects?
- What bothers you about the current structure of our classes?
- What doesn't make sense or is "ugly"?

? Question

- Do you remember our geometric objects?
- What bothers you about the current structure of our classes?
- What doesn't make sense or is “ugly”?

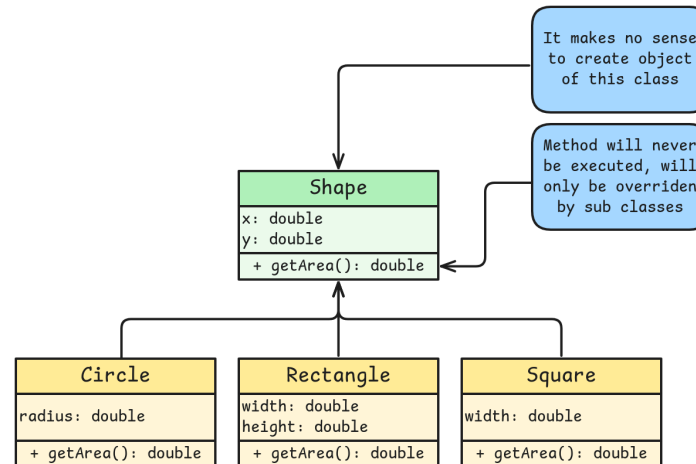


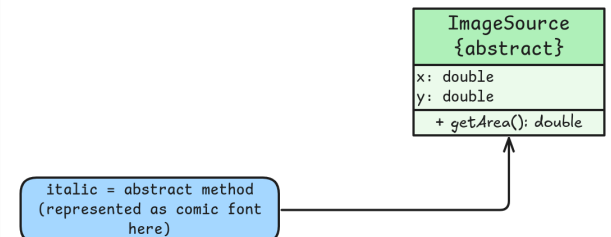
Figure 1: Overriding the method from Shape

2.1 Abstract Elements

2. Abstract Elements & Methods

- Class becomes abstract class through keyword `abstract`
- Effect: No objects of the class can be created.
- Instead:
 - ▶ Derive class and extend in (concrete = non-abstract) subclasses
 - ▶ Create objects of the subclasses

```
1  public abstract class A {  
2      // ...  
3  }  
4  
5  public class B extends A {  
6      // ...  
7  }  
8  
9  public static void main(String[] args) {  
10      A abstractObj = new A();  
11      B concreteObj = new B();  
12  }
```

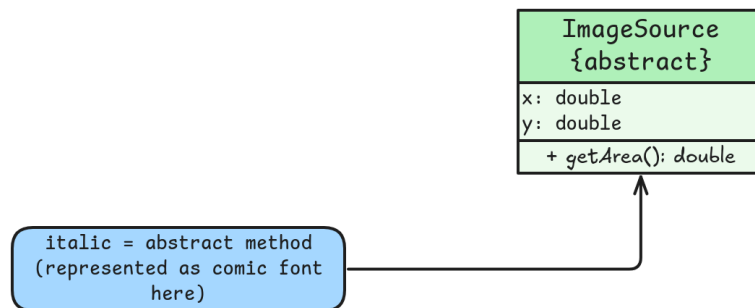


2.1 Abstract Elements

2. Abstract Elements & Methods

- Method becomes abstract method through the keyword `abstract`
- Abstract method contains only the declaration, but no implementation

```
1 public abstract class ImageSource {  
2     String name;  
3  
4     public abstract Image getNextImage();  
5 }
```



! Memorize

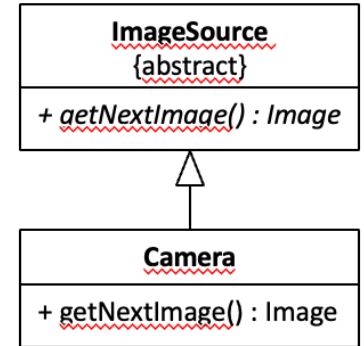
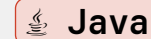
- Abstract methods cannot be called (no implementation exists!)
- Instead specifies which methods subclasses must have

2.1 Abstract Elements

2. Abstract Elements & Methods

- Classes with abstract methods must be abstract.
- Otherwise, non-implemented methods could be called for objects.
- Inheritance:
 - ▶ Abstract methods are inherited.
 - ▶ Subclasses remain abstract as long as not all abstract methods are implemented

```
1  public abstract class ImageSource {  
2      String name;  
3  
4      public abstract Image getNextImage();  
5  }  
6  
7  public class Camera extends ImageSource {  
8      public Image getNextImage() {  
9          // ...  
10     }  
11 }
```



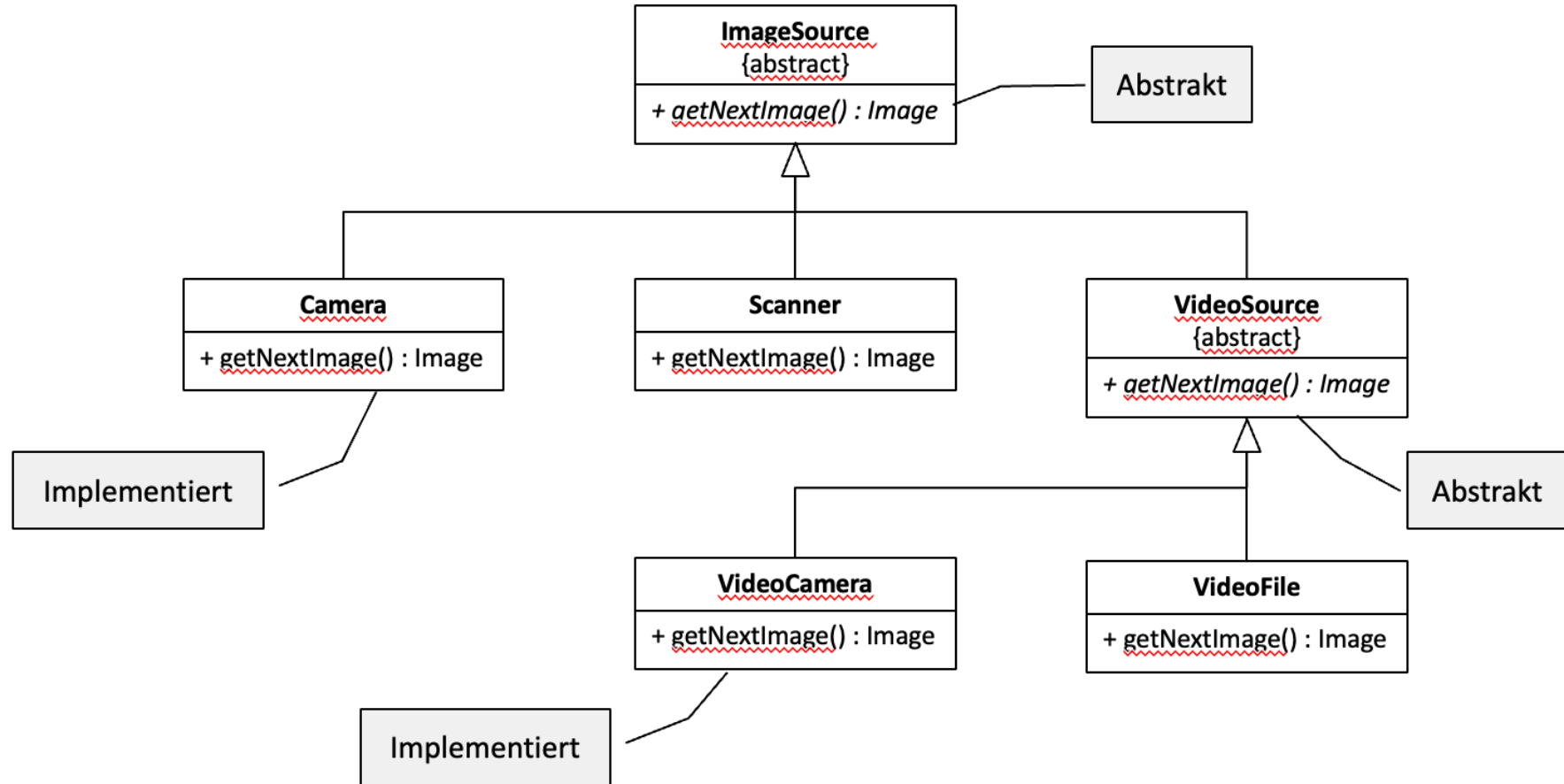
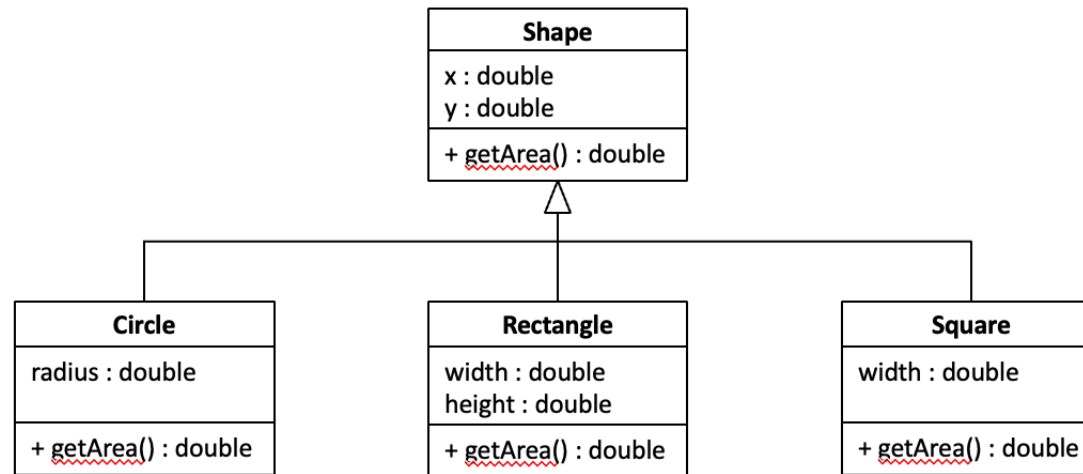


Figure 5: Large overview of abstract classes and methods

Task 1

- Now improve the structure of the class hierarchy.
- Use abstract elements for this.



2.1 Abstract Elements

2. Abstract Elements & Methods

- No objects of class Shape, but only of concrete geometric shapes
- All classes for geometric shapes have `getArea()`.
- Implementation depends on the type of geometric shape

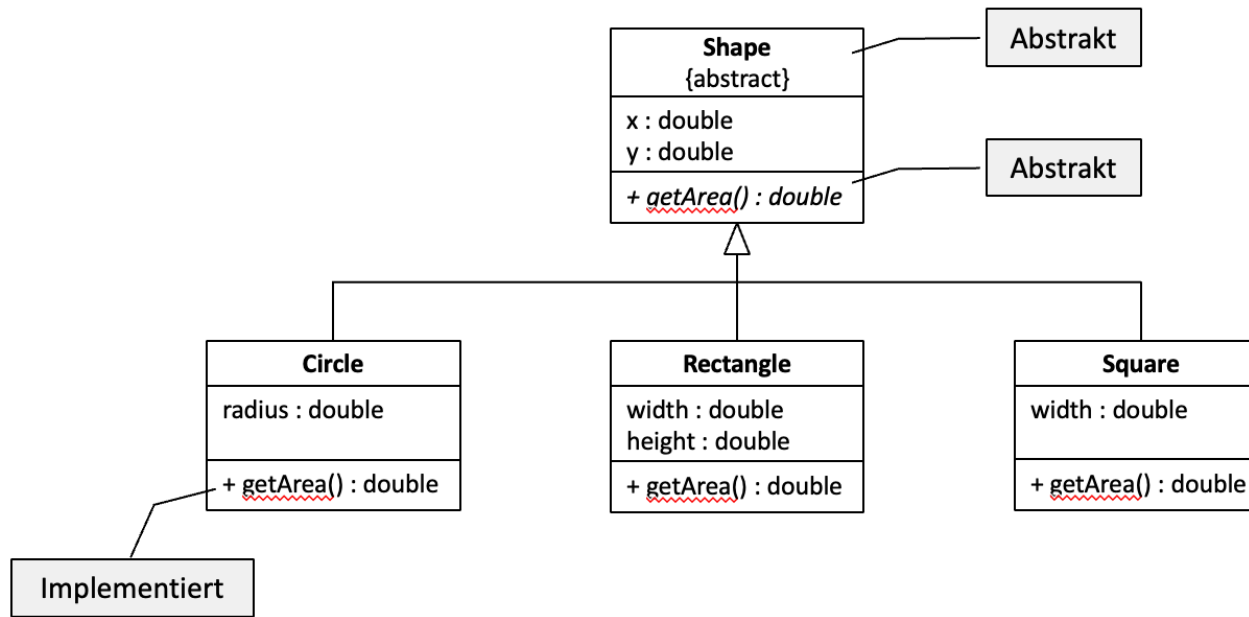


Figure 7: Abstract and implemented methods

3. Interfaces

- Classes (as a reminder):
 - ▶ Concrete classes cannot contain abstract methods.
 - ▶ Abstract classes can additionally contain abstract methods.
- Basic idea of an interface:
 - ▶ Declares only abstract methods
 - ▶ Therefore specifies which methods a class must implement
 - ▶ Contains no variables (No object creatable: no constructors needed)
 - ▶ Often describe properties (e.g. Comparable, Cloneable, Scalable, ...)

- Visibility:
 - ▶ All methods are public abstract (even when modifiers are missing).
 - ▶ All attributes are public static final (even when modifiers were missing).
- From Java 8 also implemented methods:
 - ▶ Default methods: Comparable to conventional methods in a class
 - ▶ Static methods

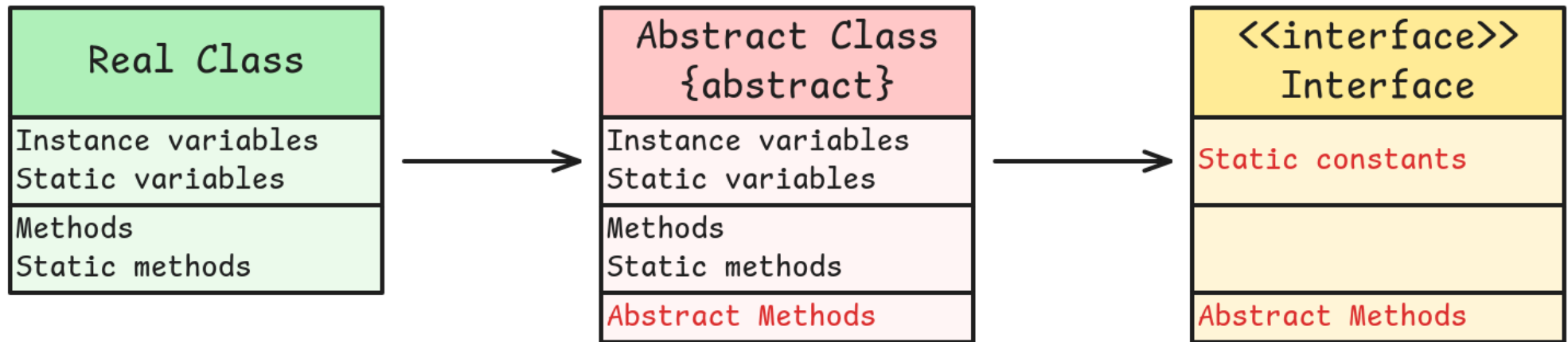


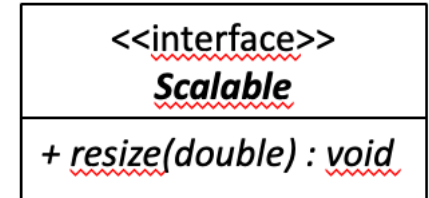
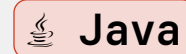
Figure 8: Gradient between Real, Abstract and Interface

3.1 Interfaces

3. Interfaces

- Declaration of an interface:

```
1  Modifier interface InterfaceName {  
2      Constants  
3      Abstract Methods  
4      Default methods and static methods  
5  }
```



- Declare method `resize()` to change size of an object

```
1  public interface Scalable {  
2      void resize(double factor);  
3  }
```

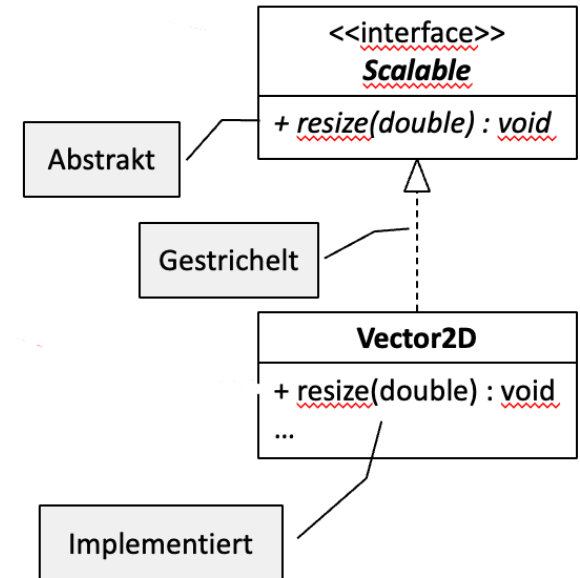


3.1 Interfaces

- Classes implement interfaces via the keyword implements
- Class inherits elements of the interface and implements abstract methods

```
1  public class Vector2D implements Scalable {
2      private double x, y;
3
4      public Vector2D(double x, double y) {
5          this.x = x;
6          this.y = y;
7      }
8
9      public void resize(double factor) {
10         x *= factor;
11         y *= factor;
12     }
13     // Additional methods ...
14 }
```

Java



- Interface method not implemented: Method remains abstract
- Therefore the class is also abstract
- Subclasses only become concrete when all abstract methods are implemented

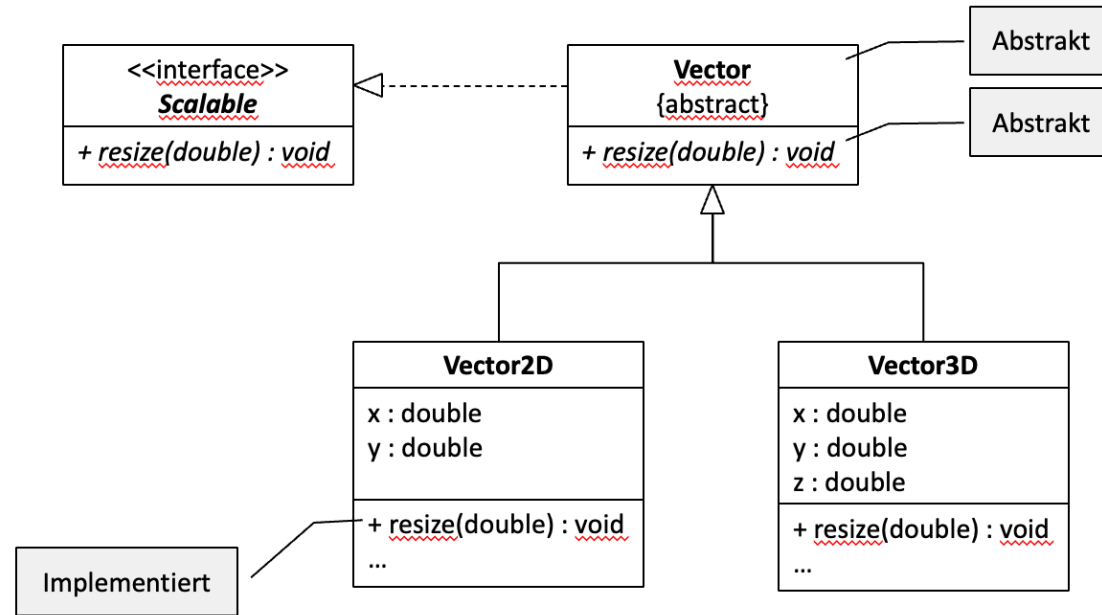
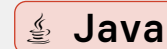


Figure 11: Abstract classes and interfaces

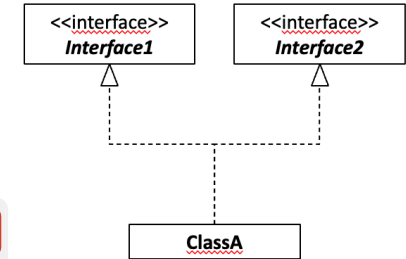
3.1 Interfaces

- As a reminder: Multiple inheritance for classes not allowed
- But: Implementation of any number of interfaces (separated by commas) allowed

```
1  interface Interface1 {  
2      // ...  
3  }  
4  
5  interface Interface2 {  
6      // ...  
7  }  
8  
9  class ClassA implements Interface1, Interface2 {  
10     // ...  
11 }
```



3. Interfaces



3.1 Interfaces

3. Interfaces

Class GrayImage implements Scalable, Drawable and Rotateable

```
1 public class GrayImage implements Scalable, Drawable, Rotateable {  
2     // Attributes and constructors  
3     // Interface methods  
4     // Additional methods  
5 }
```

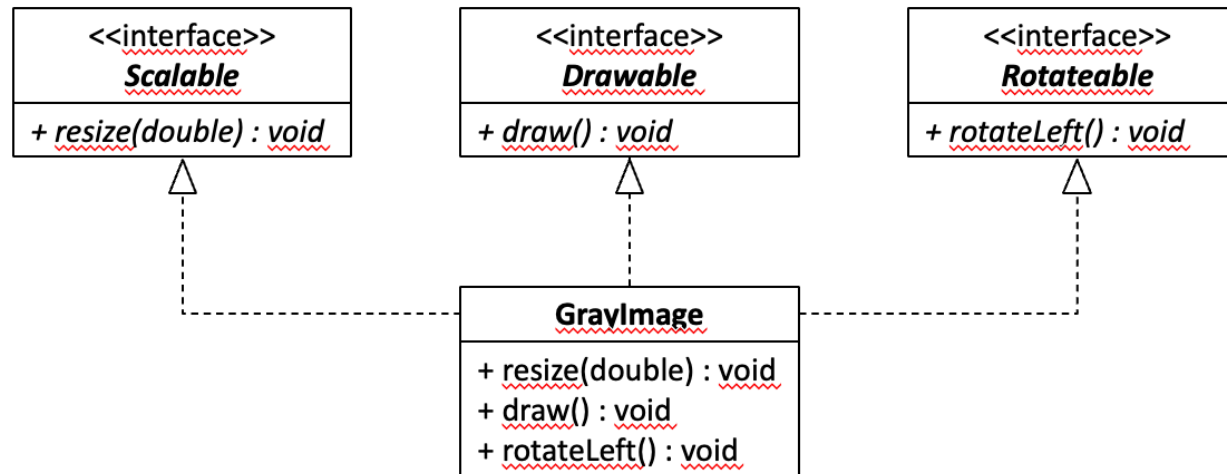
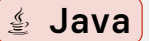


Figure 13: GrayImage-Beispiel

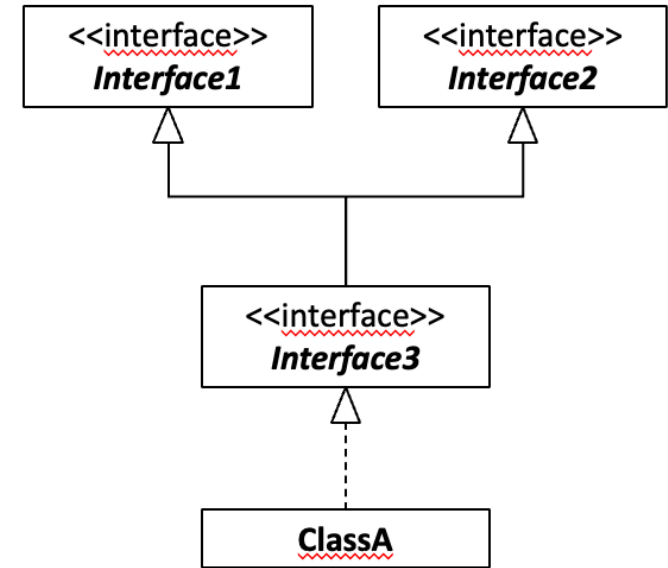
3.1 Interfaces

- Interfaces can be derived through extends.
- Multiple inheritance is allowed for interfaces!

```
1  interface Interface1 {  
2      // ...  
3  }  
4  
5  interface Interface2 {  
6      // ...  
7  }  
8  
9  interface Interface3 extends Interface1, Interface2 {  
10     // ...  
11 }  
12  
13 class ClassA implements Interface3 {  
14     // ...  
15 }
```

 **Java**

3. Interfaces



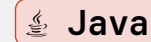
3.1 Interfaces

- Just like with base classes:
 - ▶ Objects referenceable via data types of their implemented interfaces
 - ▶ Reference variable can only access attributes and methods of its interface

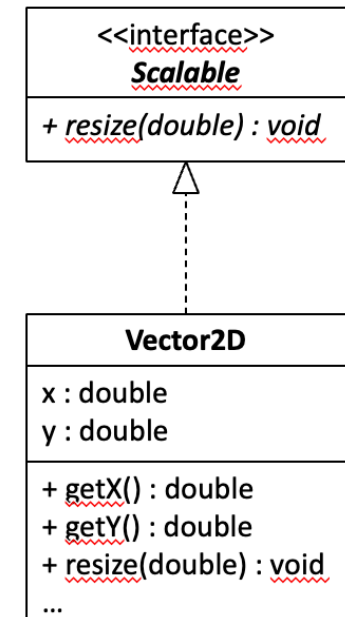
? Question

- ▶ Which accesses to attributes are allowed and which are not?

```
1  public static void main(String[] args) {
2      Vector2D classRef = new Vector2D(1, 3);
3      Scalable interRef = classRef;
4
5      classRef.resize(1.5);
6      System.out.println(classRef.getX());
7      interRef.resize(1.5);
8      System.out.println(interRef.getX());
9  }
```

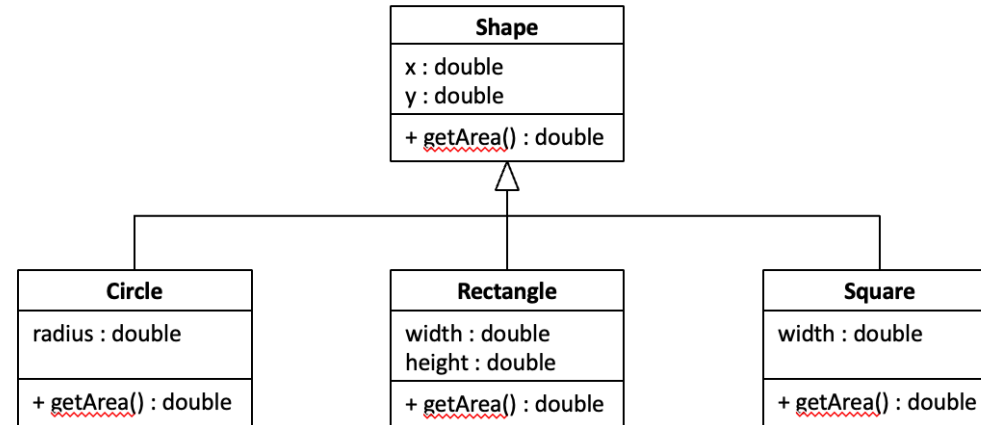


3. Interfaces



Task 2

- Create an interface Transformable with the following methods:
 - ▶ Move
 - ▶ Rotation by 90° (one method each for rotation left and right)
 - ▶ Scale
 - ▶ Implement the interface in all classes of geometric shapes



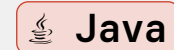
4. Comparison (Interface Comparable)

4.1 Interface Comparable

4. Comparison (Interface Comparable)

- Comparison of objects (Which is “larger”, which is “smaller”?)

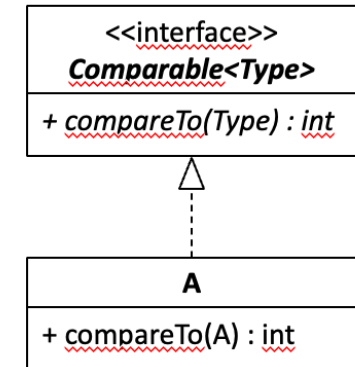
```
1 public interface Comparable<Type> {  
2     public int compareTo(Type other);  
3 }
```



- Usage:
 - ▶ Implement interface in own class
 - ▶ Replace placeholder Type with own class name
 - ▶ Return value is interpreted as follows:

Return Value	Meaning
Negative	this < other
Null	this == other
Positive	this > other

Table 1: Formats and Flags



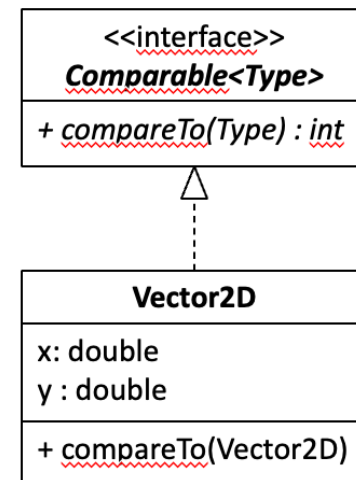
4.1 Interface Comparable

4. Comparison (Interface Comparable)

- Comparison of vectors based on magnitude:

```
1  public class Vector2D implements Comparable<Vector2D> {
2      double x, y;
3
4      public double getAbs() {
5          return Math.sqrt(x * x + y * y);
6      }
7
8      public int compareTo(Vector2D other) {
9          if (getAbs() < other.getAbs()) {
10             return -1;
11          } else if (getAbs() > other.getAbs()) {
12             return 1;
13          } else {
14             return 0;
15          }
16      }
17  }
```

 Java

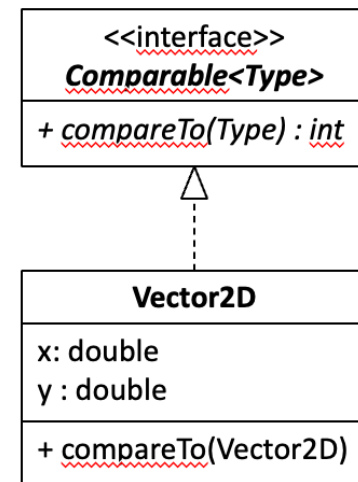


4.1 Interface Comparable

4. Comparison (Interface Comparable)

- Sort lists via class method `Collections.sort()`
- Prerequisite: Elements in list implement `Comparable`
- Method `sort()` uses the comparison method `compareTo()` pairwise

```
1  public static void main(String[] args) {
2      ArrayList<Vector2D> vectors = new ArrayList<Vector2D>();
3      vectors.add(new Vector2D(0, 5));
4      vectors.add(new Vector2D(0, -1));
5      vectors.add(new Vector2D(7, 8));
6      vectors.add(new Vector2D(0, 0));
7
8      Collections.sort(vectors);
9      for (Vector2D vector : vectors) {
10         System.out.println(vector.getAbs());
11     }
12 }
```

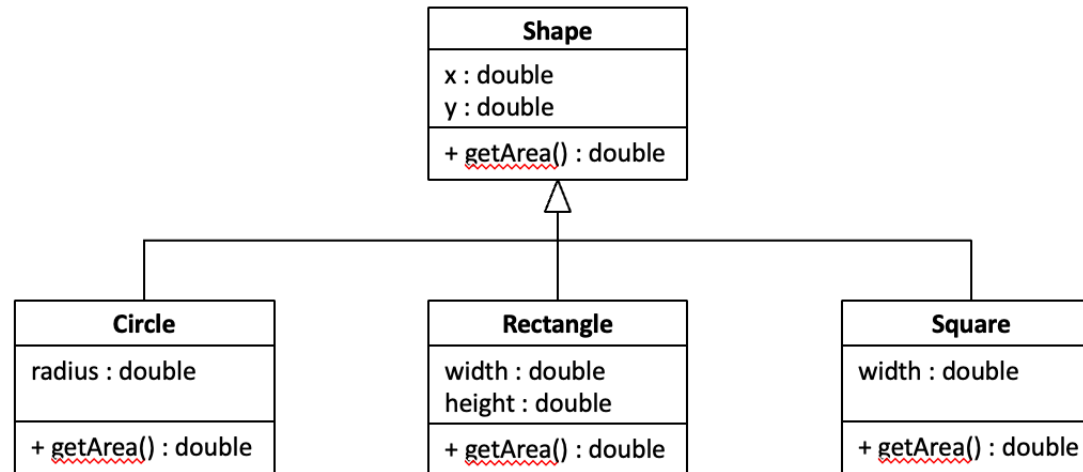


4.1 Interface Comparable

4. Comparison (Interface Comparable)

Task 3

- Implement Comparable<Type> for geometric objects.
- Criterion for the comparison is the area of the objects.

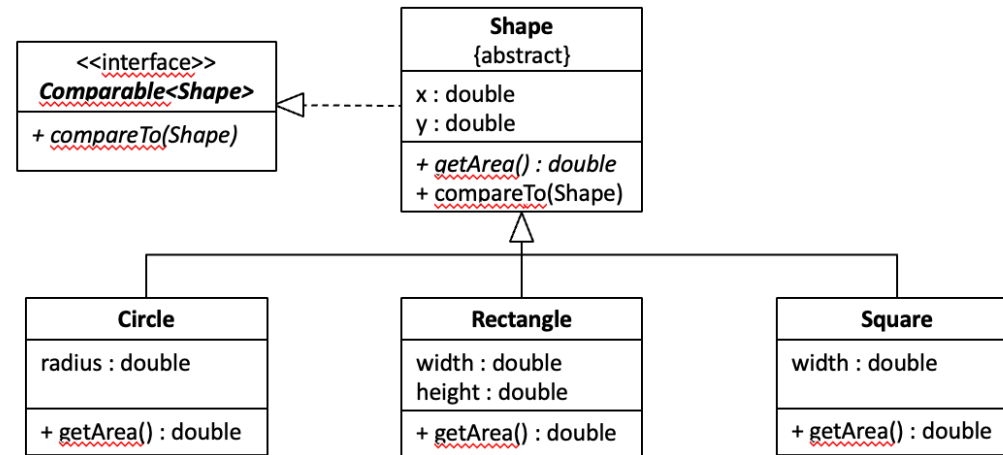


4.1 Interface Comparable

4. Comparison (Interface Comparable)

! Memorize

- Only the class Shape must implement Comparable.
- The remaining classes inherit the interface and implementation.



4.1 Interface Comparable

4. Comparison (Interface Comparable)

- Implementation in Shape:

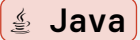
```
1  public abstract class Shape implements Comparable<Shape> {
2      // Attributes and other methods ...
3
4      public int compareTo(Shape other) {
5          double thisArea = getArea();
6          double otherArea = other.getArea();
7
8          if (thisArea < otherArea) {
9              return -1;
10         } else if (thisArea > otherArea) {
11             return 1;
12         } else {
13             return 0;
14         }
15     }
16 }
```



4.1 Interface Comparable

4. Comparison (Interface Comparable)

```
1  public static void main(String[] args) {
2      ArrayList<Shape> shapes = new ArrayList<Shape>();
3      shapes.add(new Circle(0.0, 0.0, 2.0));
4      shapes.add(new Circle(0.0, 0.0, 1.0));
5      shapes.add(new Rectangle(0.0, 0.0, 10.0, 5.0));
6      shapes.add(new Square(0.0, 0.0, 0.5));
7
8      System.out.println("Areas (unsorted):");
9      for (Shape shape : shapes) {
10         System.out.println(shape.getArea());
11     }
12
13     Collections.sort(shapes);
14     System.out.println("\nAreas (sorted):");
15     for (Shape shape : shapes) {
16         System.out.println(shape.getArea());
17     }
18 }
```



5. License Notice

5.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- <https://creativecommons.org/licenses/by-nc-sa/4.0/>
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.